生成模型：从自由能到大模型

第二周　从去噪到大模型

张士欣

中国科学院物理研究所　2026.11

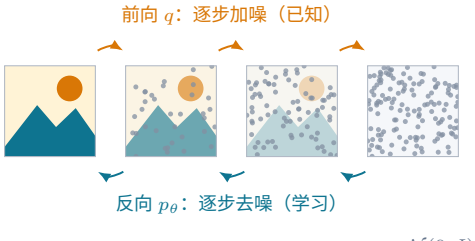
github.com/refraction-ray/free-energy-to-llms



本讲路线

上周留下的线索

- 能量模型表达最自由，却被 MCMC 采样拖累：双势阱的温度减半，链慢 40 多倍
- score $\nabla_x \log p$ 与 Z 无关，可以只用数据学
- 物理里对付势垒，靠升温、退火。扩散模型就是这么做的



本讲四部分

1. 扩散模型：在许多噪声水平上学 score，把一次难采样拆成许多小步
2. Transformer 与 KV cache：大语言模型怎样表示、怎样计算 $p(x_t \mid x_{<t})$
3. 大模型的训练阶段：从极大似然到带 KL 约束的奖励优化
4. 生成模型放回科学计算：同一个 Ising 模型的正问题与反问题；数据与评价

扩散、score 与 Flow Matching

动机：一步太难，就拆成许多小步

- GAN、VAE 想一步把噪声变成数据：映射极其复杂，或只能优化下界
- 能量模型用 Langevin 一步步走，但在多峰分布里混合极慢（上周：温度减半，慢 40 多倍）
- 扩散的思路：人为规定一个从数据走向噪声的过程，它每一步都很简单、完全已知；只学习它的反向
- 每一小步的反向转移近似是高斯，只需学一个“往哪里走”的方向

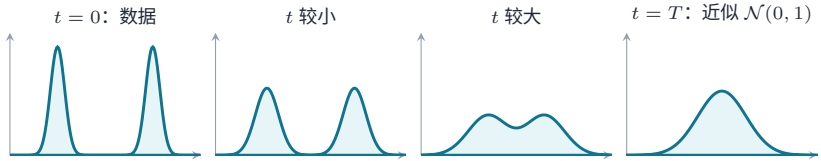
贝叶斯公式　已知前向 $q(x_t \mid x_{t-1})$，反向 $q(x_{t-1} \mid x_t) \propto q(x_t \mid x_{t-1})\, q(x_{t-1})$；难算的部分只剩带噪数据的分布。

关键想法

高噪声时分布接近高斯，各个峰连成一片，像高温；低噪声时细节丰富，但相邻两步很接近。生成就像退火：从高温慢慢降下来，每一步只修正一点。

2015	非平衡热力学视角的扩散概率模型
2019	多噪声水平的 score 模型
2020	DDPM（去噪扩散概率模型）：噪声回归，图像质量追上 GAN
2021	随机微分方程统一框架
2022–	潜空间扩散、DiT（扩散 Transformer）、Flow Matching（流匹配）

前向加噪：任意时刻都有闭式解



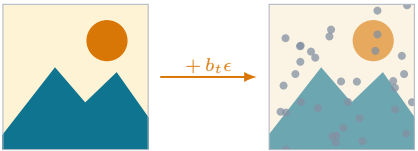
DDPM 每一步 $q(x_t \mid x_{t-1}) = \mathcal{N}(\sqrt{1-\beta_t}\, x_{t-1}, \beta_t I)$，合并后

$$x_t = \underbrace{\sqrt{\bar{\alpha}_t}}_{a_t}\, x_0 + \underbrace{\sqrt{1-\bar{\alpha}_t}}_{b_t}\, \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

其中 $\bar{\alpha}_t = \prod_{s \le t}(1-\beta_s)$。训练时直接抽 t, ϵ 合成 x_t，无需逐步模拟。

高斯卷积　独立高斯之和仍是高斯，方差递推 $v_t = \alpha_t v_{t-1} + \beta_t$ 解得 $1 - \bar{\alpha}_t$。带噪分布 p_t 是数据分布与高斯核的卷积：峰被抹平、最终只剩一个高斯。

想一想：猜出加进去的噪声



训练时只把 x_t 和 t 交给网络，原图 x_0 不给。

问题

让网络从 x_t 说出加进去的噪声 ϵ。

1 这件事做得到吗？
2 做不到的话，用均方误差训练，网络会学到什么？学到的东西有什么用？

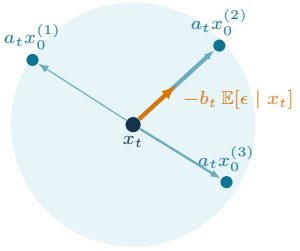
训练：让网络猜加进去的噪声

训练：噪声回归

1. 取数据 x_0，随机抽 t 与 $\epsilon \sim \mathcal{N}(0, I)$
2. 合成 $x_t = a_t x_0 + b_t \epsilon$
3. 最小化 $\|\epsilon - \epsilon_\theta(x_t, t)\|^2$

网络输入 x_t 与 t，所有 t 共用一个网络（U-Net 或 Transformer）。猜出噪声即猜出原图：$\hat{x}_0 = (x_t - b_t \epsilon_\theta)/a_t$。

回到问题 第 1 问：做不到，同一个 x_t 有多种可能的来历。均方回归的最优解是条件期望 $\mathbb{E}[\epsilon \mid x_t = x]$，网络学到的是后验平均（右图橙色）。它有什么用？见下一页。



同一个 x_t 可能来自不同原图。青色：若原图是 $x_0^{(k)}$，去噪方向是 $-b_t \epsilon^{(k)}$，粗细表示后验 $p(x_0^{(k)} \mid x_t)$；橙色：网络学到的后验平均

为什么预测噪声就是在学 score

★ 关键一步

带噪分布是高斯混合，$q(x \mid x_0) = \mathcal{N}(a_t x_0, b_t^2 I)$：

$$p_t(x) = \int p_{\text{data}}(x_0)\, q(x \mid x_0)\, dx_0$$

对 x 求梯度，并利用 $\nabla q = q \nabla \log q$：

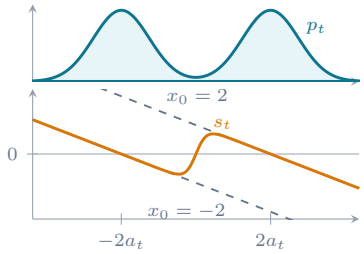
$$\nabla \log p_t(x) = \int \underbrace{\frac{p_{\text{data}}(x_0)\, q(x \mid x_0)}{p_t(x)}}_{\text{后验} p(x_0 \mid x_t = x)} \nabla \log q \, dx_0$$

$$= \mathbb{E}\Big[-\frac{x - a_t x_0}{b_t^2} \;\Big|\; x_t = x \Big]$$

$$= -\frac{1}{b_t}\, \mathbb{E}[\epsilon \mid x_t = x]$$

即各张原图的 score 按后验加权平均（用了 $x - a_t x_0 = b_t \epsilon$）。对照上一页：

$$s_t(x) \approx -\epsilon_\theta(x, t)/b_t$$



数据只有 $x_0 = \pm 2$ 两点（$a_t = 0.8$，$b_t = 0.6$）。虚线：两张原图各自的条件 score；橙色：总 score，按后验权重在两者之间切换

回到问题　这个后验平均恰好就是 score。不用算 Z，也没有散度项，只做去噪回归。

采样：从噪声一步步走回数据

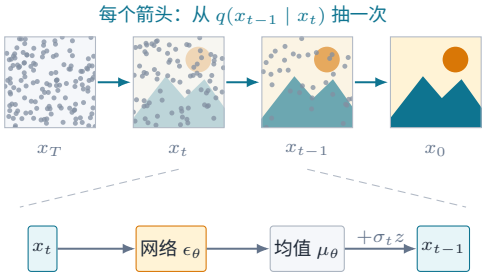
前向是一条已知的马尔可夫链 $x_0 \to x_1 \to \cdots \to x_T$。采样就是倒着走：每一步从反向转移 $q(x_{t-1} \mid x_t)$ 抽一个样本。

采样：逐步去噪

1. $x_T \sim \mathcal{N}(0, I)$
2. 对 $t = T, \ldots, 1$：抽 $z \sim \mathcal{N}(0, I)$（$t = 1$ 时取 $z = 0$），

$$x_{t-1} = \underbrace{\frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{b_t}\epsilon_\theta(x_t, t)\right)}_{\text{均值}\mu_\theta(x_t)} + \sigma_t z$$

3. 输出 x_0。共调用网络 T 次（DDPM 中 $T = 1000$）



上：生成方向，噪声逐步变成图像。下：放大其中一步

高斯近似　步长 β_t 很小时，反向一步近似高斯，只需知道均值和方差。方差 σ_t^2 由噪声调度定好；均值由网络算出，公式从哪来见下一页。

反向一步：先猜原图，再朝它挪一小步

★ 关键一步

假如知道原图 x_0：由贝叶斯公式

$$q(x_{t-1} \mid x_t, x_0) \propto \underbrace{q(x_t \mid x_{t-1})}_{\text{前向一步}} \; \underbrace{q(x_{t-1} \mid x_0)}_{\text{闭式加噪}}$$

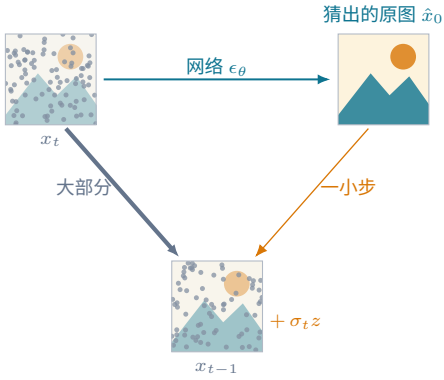
两个因子都是 x_{t-1} 的高斯，乘积仍是高斯，均值是 x_t 与 x_0 的加权组合：

$$\tilde{\mu}_t = \underbrace{\frac{\sqrt{\alpha_t}\, b_{t-1}^2}{b_t^2}}_{\approx 1} x_t + \underbrace{\frac{a_{t-1}\beta_t}{b_t^2}}_{\text{很小}} x_0, \qquad \tilde{\beta}_t = \frac{b_{t-1}^2}{b_t^2}\,\beta_t$$

不知道 x_0：代入网络的估计 $\hat{x}_0 = (x_t - b_t\epsilon_\theta)/a_t$，化简得：

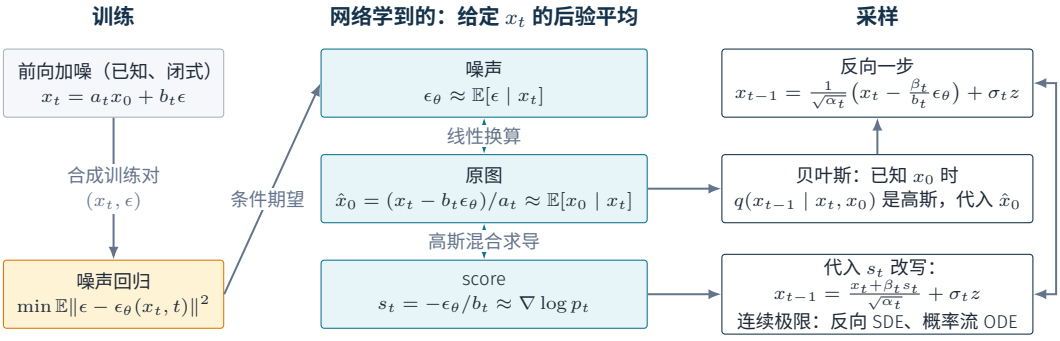
$$\mu_\theta(x_t) = \frac{1}{\sqrt{\alpha_t}}\Big(x_t - \frac{\beta_t}{b_t}\,\epsilon_\theta(x_t, t)\Big), \qquad \sigma_t^2 = \tilde{\beta}_t$$

这正是上一页算法里的均值。$\tilde{\mu}_t$ 对 x_0 线性，由全期望公式，代入 $\mathbb{E}[x_0 \mid x_t]$ 恰好得到 $\mathbb{E}[x_{t-1} \mid x_t]$。



每一步：网络先从 x_t 猜出原图 $\hat{x}_0$（只是后验平均，比较模糊），再把 x_t 朝 $\hat{x}_0$ 挪一小步，加一点噪声。猜得粗没关系，下一步会在更清楚的图上重新猜

串起来：一个网络，三种读法

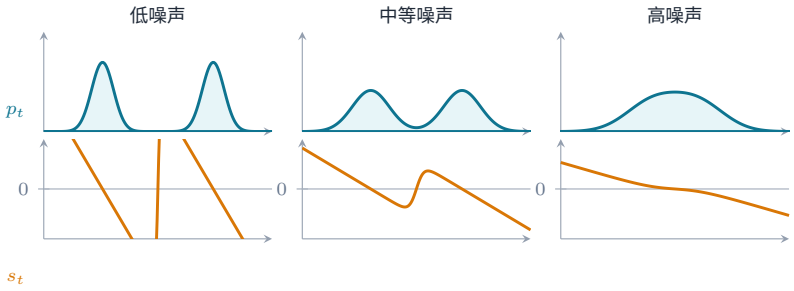


用到的概率工具：高斯卷积（前向）　条件期望（训练）　混合分布的梯度（score）　贝叶斯公式与全期望公式（采样）

要点

噪声、原图、score 只差线性换算，都是给定 x_t 的后验平均。训练只用条件期望；采样一步由贝叶斯公式推出，换成 score 写，连续极限就是后面的 SDE 与 ODE。

不同噪声水平的 score



要点

低噪声时 score 精细而陡峭，只在数据附近可靠；高噪声时 score 平滑，几乎是指向原点的线性场。采样从高噪声出发，先定大致位置，再逐级补细节，这避开了单一 score 在数据远处学不准、混合慢的问题。

反向扩散与 Langevin：像在哪，不同在哪

★ 换成 score 的写法

代入 $\epsilon_\theta = -b_t s_t$，采样的一步是

$$x_{t-1} = \frac{x_t + \beta_t\, s_t(x_t)}{\sqrt{\alpha_t}} + \sigma_t z$$

上周的 Langevin 是

$$x \leftarrow x + \tfrac{\eta}{2}\, s(x) + \sqrt{\eta}\, z$$

形式相同：沿 score 走一步，再加高斯噪声。

拆开看　步长小时 $\sigma_t^2 \approx \beta_t$。忽略 $\sqrt{\alpha_t}$ 的缩放，一步的增量拆成两半：

$$\beta_t s_t + \sqrt{\beta_t}\, z = \underbrace{\tfrac{1}{2}\beta_t s_t + \sqrt{\beta_t}\, z}_{\eta=\beta_t \text{ 的 Langevin}} + \underbrace{\tfrac{1}{2}\beta_t s_t}_{\text{确定性输运}}$$

前一半只在 p_t 内部搅动，不改变分布；后一半把 p_t 推向 p_{t-1}，就是下一页的概率流 ODE。

	Langevin	反向扩散
目标	一个固定的 p	一串分布 $p_T \to p_0$
score	每步相同	每步换成 s_t
每一步	保持 p 不变	把 p_t 变成 p_{t-1}
步数	等链混合，事先不知	正好 T 步
多峰	卡在一个峰里	高噪声时峰已连通

要点

Langevin 在一个固定分布里反复游走，要等它混合；反向扩散每一步都换一个分布，顺着 $p_T \to \cdots \to p_0$ 走下来，走完 T 步就是数据的样本。

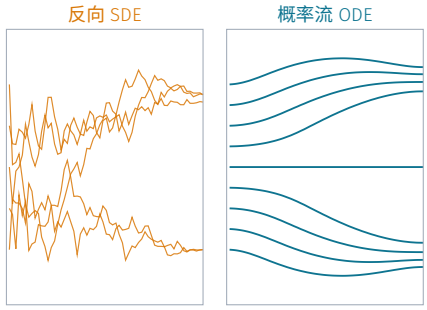
连续时间：随机微分方程与概率流

正向加噪写成随机微分方程（SDE）：$dx = f(x,t)\,dt + g(t)\,dW_t$。

- 反向 SDE（随机）：$dx = \big[f - g^2\, s_t(x)\big]dt + g\, d\bar{W}_t$
- 概率流 ODE（确定）：$\dfrac{dx}{dt} = f - \frac{1}{2} g^2\, s_t(x)$

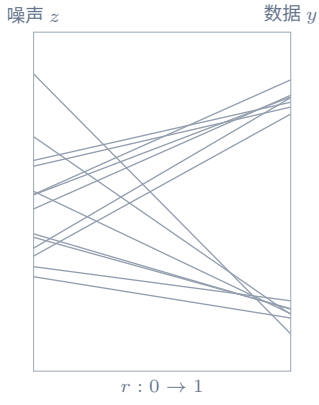
两者在每个时刻的边缘分布 p_t 都相同，单条轨迹不同。缺的只有 score，正好由噪声网络给出。

Fokker–Planck 方程　密度演化中的扩散项可以改写成输运：$\partial_t p = -\nabla \cdot \big[(f - \frac{1}{2} g^2 s)\, p\big]$，这就是概率流 ODE 的来历。



横轴为生成时间。两种轨迹都落到两个模式上，ODE 轨迹互不相交

想一想：直线训练，学出什么轨迹



另一种训练方式 Flow Matching（流匹配）：随机配对噪声 z 与数据 y，连成直线 $x_r = (1-r)z + ry$；在直线上任取一点，让网络回归这条直线的速度 $y - z$。

问题

训练用的全是直线，而且互相交叉。用学到的速度场生成时，轨迹是什么样？

A 直线，会交叉
B 直线，互不相交
C 弯曲，互不相交

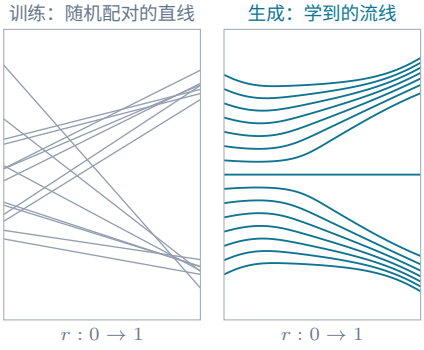
Flow Matching：直接学速度

取噪声 z 与数据 y，走直线 $x_r = (1-r)z + ry$，它的速度是 $y - z$：

$$\mathcal{L} = \mathbb{E}_{y,z,r}\left\| v_\theta(x_r, r) - (y - z) \right\|^2$$

- 同一个 x_r 可能来自许多对 (z, y)；最优解是条件平均 $v_r(x) = \mathbb{E}[y - z \mid x_r = x]$
- 训练只需随机取一个时刻做回归，不解任何 ODE
- 生成：$z \sim \mathcal{N}(0, I)$，沿 $dx/dr = v_\theta$ 积分到 $r = 1$

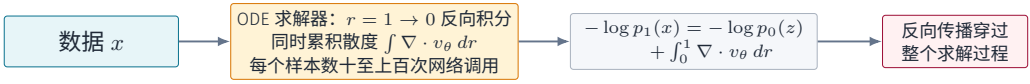
条件期望　和噪声回归同一个原理：对单对样本的回归，学到总体的速度场。



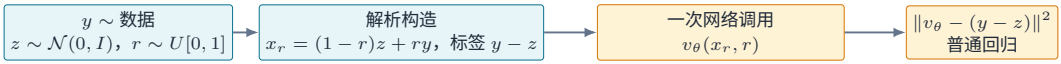
回到问题　C。ODE 的解唯一，流线不能相交。在训练直线交叉的地方，网络只能输出各条直线速度的平均，流线因此弯开。

Flow Matching 与连续归一化流（CNF）：同一个模型，两种训练

CNF：极大似然训练



Flow Matching：回归训练



推理（两者完全相同）：$z \sim \mathcal{N}(0, I)$，数值积分 $dx/dr = v_\theta(x, r)$ 得到样本；需要密度时同样沿轨迹积分散度

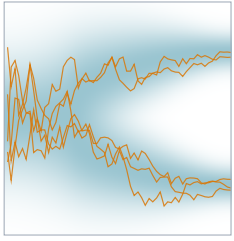
要点

训练从“每个样本解一次 ODE、积分散度、对整条轨迹反传”变成“随机取一个时刻、做一次回归”；学到的速度场和生成 ODE 与 CNF 完全相同。要学一个把高斯搬运成数据的速度场，训练时却不必模拟这个搬运；连续时间流因此能扩展到图像、视频等高维数据。

全景：一条概率之河

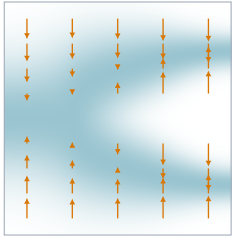
这几种方法做的是同一件事：选一条从高斯连续变到数据的概率路径 p_t，学一个向量场，让样本顺着它走回数据。下图横轴是生成时间（左为噪声，右为数据），纵轴是 x，颜色是 $p_t(x)$。

1 加噪：河由数据吹散而成



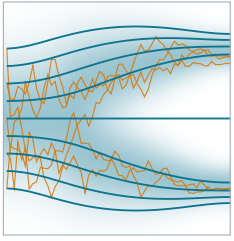
从数据向噪声扩散，已知

2 score：河面的坡度



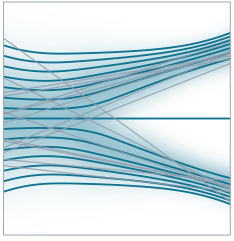
箭头 $\nabla \log p_t$ 指向密度高处

3 扩散采样：逆流走回



橙：反向 SDE；
青：概率流 ODE

4 Flow Matching：换一条直河



灰：训练直线；青：学到的流线

	河（概率路径）	网络学什么	怎么走回数据
扩散	逐步加噪 $x_t = a_t x_0 + b_t \epsilon$	score，即河面的坡度	反向 SDE 或概率流 ODE
Flow Matching	直线插值 $x_r = (1-r)z + ry$	速度，即水流本身	沿速度场积分 ODE

扩散：小结

概率对象

各噪声水平的 score $\nabla_x \log p_t$
（或噪声、速度）

训练目标

噪声 / score / 速度的均方回归
负 ELBO 的重加权

采样方式

从高斯出发，反向 SDE 或 ODE
数十到上千次网络调用

代价与失效

采样步数多；似然需另行积分

要点

扩散绕开了 Z（只学梯度），绕开了混合慢（噪声把各模式连通），也避开了对抗博弈（训练只是回归）。代价是采样要多步。文本也可以做“掩码扩散”：前向把 token 逐步换成 [MASK]，反向逐步填回，这与下面 BERT 的完形填空相连。

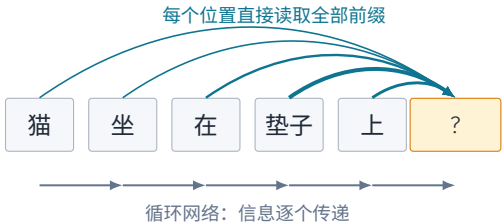
Transformer：怎样表示 $p(x_t \mid x_{<t})$

动机：需要一个读前缀、出分布的网络

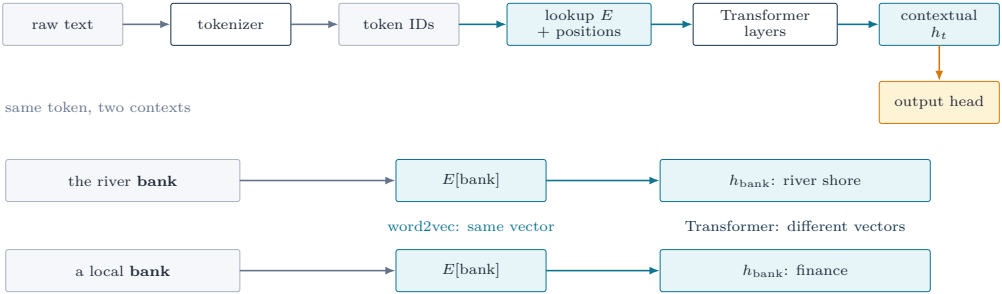
自回归模型要一个函数：输入任意长的前缀 $x_{<t}$，输出词表上的分布。

- **n-gram**：只看前 $n-1$ 个 token，查频率表；上下文短，表随 n 指数增长
- **循环网络**：把历史压进一个向量，逐个更新；训练也只能串行，长距离信息易丢失
- **Transformer**（2017）：每个位置直接按内容读取所有可见位置；训练时所有位置并行

马尔可夫假设　n-gram 是 $n-1$ 阶马尔可夫链；Transformer 不做截断，条件是整个前缀。



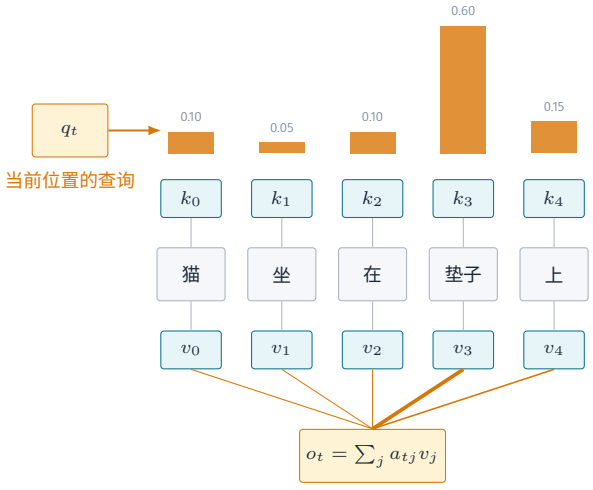
从文字到向量



- 分词器把文字切成 token 编号；编号只是类别标签，大小没有含义
- 嵌入矩阵 $E \in \mathbb{R}^{|\mathcal{V}| \times d}$ 为每个编号查一行向量；它与模型一起由训练损失更新
- 查表向量与上下文无关；经过 Transformer 层后，同一个 token 在不同句子中得到不同的表示

注意力：按内容加权平均

$$a_{tj} = \mathrm{softmax}_j\left(q_t \cdot k_j / \sqrt{d_k}\right)$$

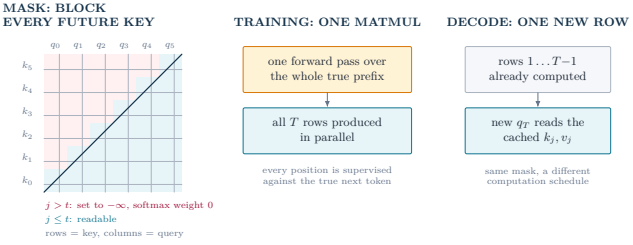


- query q：当前位置“想找什么”
- key k：每个位置“按什么被检索”
- value v：被读取的内容
- 三者都是同一表示经不同矩阵投影：$Q = XW_Q$，$K = XW_K$，$V = XW_V$

期望　权重 $a_{t\cdot}$ 是位置上的一个 categorical 分布；输出 o_t 是 value 在这个分布下的期望。softmax 又是玻尔兹曼分布：$-q \cdot k$ 扮演能量。

缩放点积注意力与因果 mask

$$\mathrm{Attn}(Q,K,V)=\mathrm{softmax}\Big(\frac{QK^{\top}}{\sqrt{d_k}}+M\Big)V,\qquad M_{tj}=0\ (j\le t),\quad M_{tj}=-\infty\ (j>t)$$



因果 mask 让位置 t 只看见 $x_{\le t}$：一次矩阵乘法给出所有位置的预测，标签不会泄漏。

独立和的方差　q,k 各分量独立、均值 0、方差 1 时 $\mathrm{Var}(q\cdot k)=d_k$；除以 $\sqrt{d_k}$ 让 logit 方差为 1，softmax 不会过早饱和。

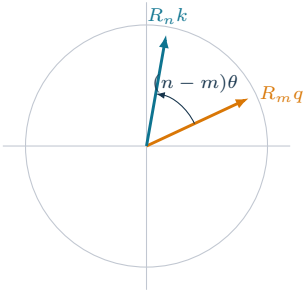
多头与位置

多头注意力

$$\mathrm{MHA}(X) = \mathrm{Concat}(\mathrm{head}_1, \ldots, \mathrm{head}_h)\, W_O$$

每个 head 有自己的 W_Q, W_K, W_V，在各自子空间里用不同的相似度检索：有的看语法搭配，有的看指代，有的看相邻位置。W_O 把各 head 的结果混合回残差流。

位置信息　不加 mask 的注意力只看内容：打乱 token 顺序，输出只是跟着重排（置换等变）。所以要另外注入位置。



RoPE（旋转位置编码）：把 q, k 每两维看作平面向量，在位置 m 旋转角度 $m\theta$。内积 $(R_m q)^\top (R_n k) = q^\top R_{n-m} k$ 只依赖相对位置 $n - m$。

想一想：去掉位置编码

上一页：不加 mask 的注意力只看内容。没有位置编码的双向模型（如 BERT）里，每个词都读到同样三个词，“狗”的表示在“狗咬人”和“人咬狗”中完全一样，分不清谁咬谁。

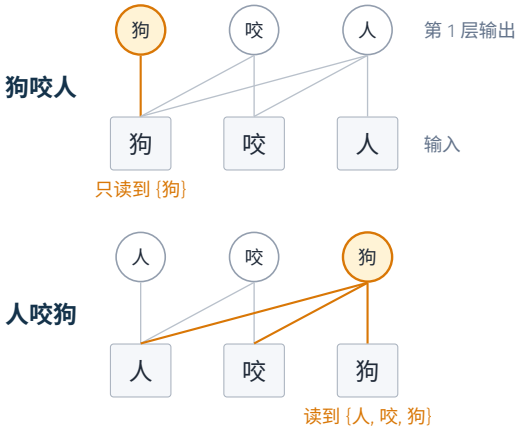
现在换成 GPT：因果 mask 照旧，只把位置编码整个去掉，其余不变。

问题

没有位置编码的 GPT，分得清“狗咬人”和“人咬狗”吗？

A 分不清：注意力只看内容
B 分得清

因果 mask 自带顺序

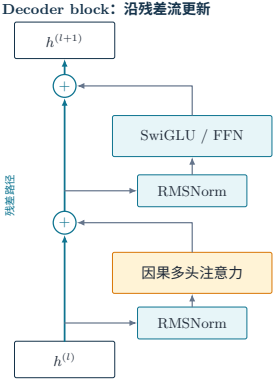


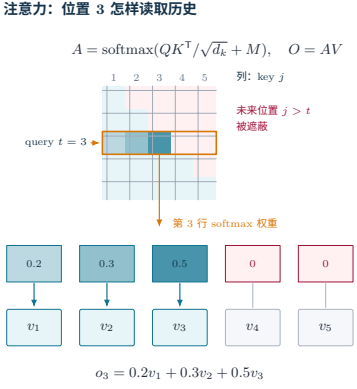
连线：因果 mask 下每个位置能读的位置。橙色：“狗”的表示从哪里来

回到问题 B。因果 mask 下，同一个词站在不同位置，能读到的集合不同：“狗咬人”里的“狗”在最前面，只读到自己；“人咬狗”里的“狗”在最后，读到全部三个词。一层之后，“狗”的表示在两句里就不同了，模型由此知道它在前还是在后。

对称破缺 因果 mask 不只是防止偷看后文，它破坏了置换对称，把顺序隐式写进了网络。不加位置编码（NoPE）的语言模型照样学得到位置（Haviv 等，2022），在一些长度外推实验中还更好（Kazemnejad 等，2023）。Kimi Linear 的全注意力层就不加位置编码，把位置信息交给线性注意力层。

一个 Transformer block

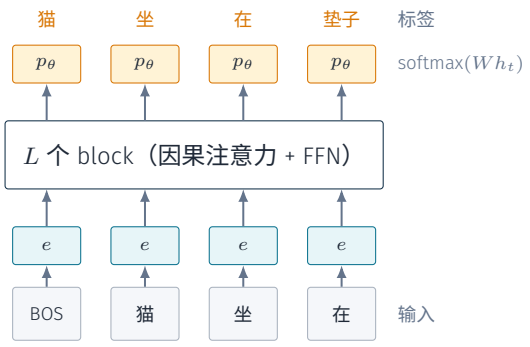




- **残差流**：$h \leftarrow h + \mathcal{F}(\mathrm{Norm}(h))$，每个子层只写入一个增量，深层也能稳定训练
- **注意力**在位置之间交换信息
- **FFN**（前馈网络）在每个位置内部做非线性特征变换
- **归一化**把每个 token 的向量调到稳定尺度

一个 block 堆叠 L 次（几十到上百层）。

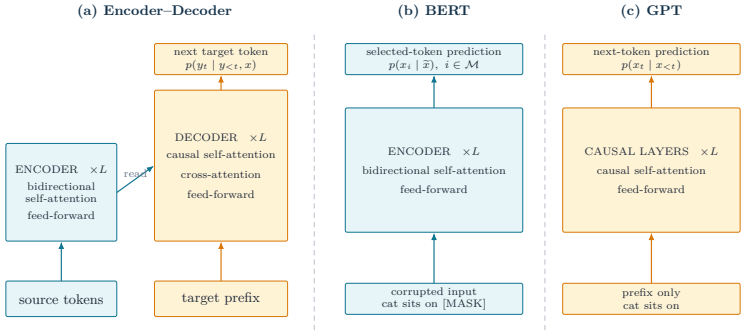
从最后一层到下一个 token 的分布



$$p_\theta(x_{t+1} = v \mid x_{\le t}) = \mathrm{softmax}\big(W h_t^{(L)}\big)_v$$

- **训练**：整段真实文本一次送入，每个位置都与下一个 token 计算交叉熵，即上周的极大似然
- **生成**：抽出一个 token 接到末尾，再算下一个位置；必须串行
- **KV cache**（键值缓存）：前缀各位置的 k, v 不会再变，生成时缓存起来，每步只算新位置（下一节）

三种结构，三种条件分布



结构	学习的分布	用途
Encoder–Decoder	$p(y_t \mid y_{<t}, x)$	翻译等“读完再写”的任务
BERT	$p(x_i \mid \tilde{x})$：遮住一些词再猜回	双向编码器：文本表示、分类、检索
GPT	$p(x_t \mid x_{<t})$：预测下一个 token	生成式预训练：续写、对话、大语言模型

KV cache：生成一个 token 要算什么

想一想：越聊越慢

一次长对话（或一个 Agent 任务）已经积累了 10^5 个 token 的上下文。模型每次只多生成一个 token：读入前缀，算出 $p_\theta(x_{t+1} \mid x_{\le t})$，抽样，接到末尾。

提示：因果 mask 下，旧位置看不到新加入的 token。

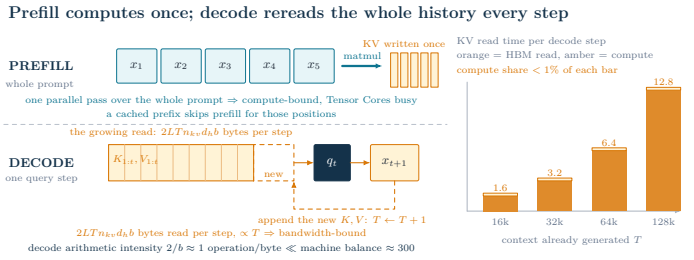
问题

生成第 $10^5 + 1$ 个 token 的计算量，与生成第 1 个 token 相比？

A 差不多
B 正比于上下文长度
C 正比于上下文长度的平方

KV cache：用存储换重算

回到问题　B，前提是缓存。旧位置看不到新 token，它们在每一层的 key 和 value 都不会再变，可以存下来（KV cache）。每生成一个 token，只为新位置算各层投影和 FFN，再用新的 query 把全部历史 K/V 读一遍：读取量正比于上下文长度。每步都重算整个前缀，注意力部分就是平方。



Prefill：已知的 prompt 一次并行算完，写满缓存；大矩阵乘法，受算力限制。

Decode：逐 token 生成，每步把全部 K/V 从显存读一遍；受显存带宽限制，算力大多闲着。

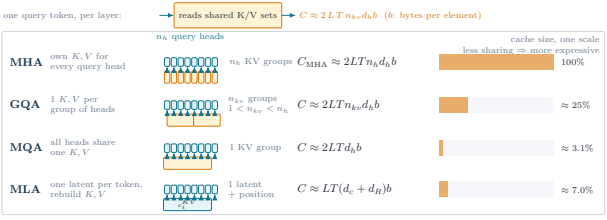
所以输出 token 比输入 token 贵，命中缓存的输入更便宜；改动对话开头的一个字，其后的缓存全部作废。

KV cache 有多大

$$C_{\mathrm{KV}} \approx 2\,L\,T\,n_{kv}\,d_h\,b$$

key、value 各一份；L 层；上下文长度 T；每层 n_{kv} 组 K/V，每组 d_h 维；每个元素 b 字节。

例：$L = 80$，$n_{kv} = 8$，$d_h = 128$，$T = 128\,000$，FP16（$b = 2$），单个请求约 **39 GiB**，接近一张 80 GB GPU 显存的一半，而且每生成一个 token 都要整个读一遍。



> **要点**
>
> 让多个 query head 共享一组 K/V（GQA、MQA），或每个 token 只缓存一份低维摘要、读取时再展开（MLA），都在缩小这个数。代价是各 head 能用的 K/V 子空间变少。

DeepSeek 与 Kimi：长上下文的几种改法

模型	改动	做法	代价
DeepSeek-V2 (2024)	MLA	每个历史 token 只缓存一份低维摘要，读取时再展开成各 head 的 K/V	压缩会丢细节
DeepSeek-V3.2 (2025)	稀疏注意力 DSA	轻量索引器先给全部历史打分，主注意力只精读得分最高的 k 个位置	索引器要单独训练；选漏了就看不到
Kimi Linear (2025)	线性注意力 KDA + MLA	三层线性注意力维护固定大小的状态（工作记忆），每隔三层插一层 MLA 翻查完整历史	固定状态会把旧关联挤在一起
DeepSeek-V3 (2024)	MoE（混合专家）	FFN 换成数百个专家，每个 token 只激活其中几个：总参数 671B，每个 token 只用 37B	路由与负载均衡增加训练复杂度

> **要点**
>
> 前三项改的都是“历史怎样保存和读取”：缓存更小、读得更少、或用固定状态代替全部历史；第四项改的是每个 token 用多少参数。长上下文正是 Agent 这类长任务最吃资源的地方。

大模型的训练阶段

从续写到对话，再到推理

1. 预训练：学会续写

预测下一个 token

`Q: 1+1? A: 2`

`Q: 3+3? A:`

数万亿 token 的网页、书籍、代码

2. 对齐：学会回答

SFT（监督微调）：示范“合格回答”

RLHF（基于人类反馈的强化学习）：人类比较两个回答的好坏

用户：提问　助手：回答

3. 推理：学会多想

RLVR（可验证奖励的强化学习）：程序判定答案对错，奖励正确的解题过程

`<think>…</think>`

数学、代码等可验证任务

要点

三个阶段用的是同一个自回归模型 $\prod_t p_\theta(x_t \mid x_{<t})$，改变的是：经验分布由谁提供，以及目标里有没有奖励。

预训练与 SFT：同一个极大似然

预训练　在语料经验分布 $\hat{\pi}$ 上最小化负对数似然（NLL），即 $\mathrm{KL}(\hat{\pi}\|p_\theta)$

- 数据配比 $\hat{\pi}=\sum_k \alpha_k \hat{\pi}_k$（网页、代码、数学、多语言……）本身就是在选择目标分布
- 参数量、数据量、算力需要按 scaling law 配平

SFT　换成“指令 c + 合格回答 y”的数据，损失只在回答 token 上计算：

$$\mathcal{L}_{\mathrm{SFT}} = -\mathbb{E}_{(c,y)} \sum_t \log p_\theta(y_t \mid y_{<t}, c)$$

经验分布与混合分布　预训练和 SFT 的差别只在 $\hat{\pi}$：前者覆盖所有文本，后者集中在“回答问题”这一小部分。

SFT 的局限　只有正例：模型学到“这样回答可以”，却学不到“另一种回答差多少”。比较好坏需要奖励或偏好。

RLHF：在参考分布附近提高奖励

给回答打分的奖励 $r(x)$ 来自人类偏好训练的奖励模型或程序判题器。目标：提高奖励，又不远离参考模型 p_0（通常是 SFT 模型）。

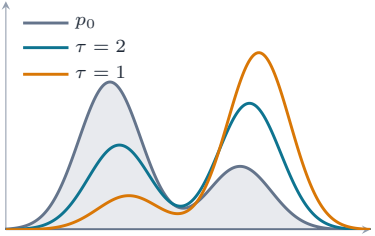
★ 关键一步

$$\mathbb{E}_q[r] - \tau \mathrm{KL}(q\|p_0) = -\tau\, \mathbb{E}_q\left[\log q - \log p_0 - \tfrac{r}{\tau}\right]$$

$$= -\tau\, \mathrm{KL}(q\|q^*) + \tau \log Z$$

所以最优策略是 $q^* = p_0\, e^{r/\tau}/Z$。

玻尔兹曼分布与自由能　奖励是负能量，τ 是温度，p_0 是先验测度；目标就是上周的变分自由能。



回答空间（示意）

奖励 $r(x) = x$：τ 越小，概率越向高奖励一侧移动；但只能在 p_0 已有质量的地方重新分配

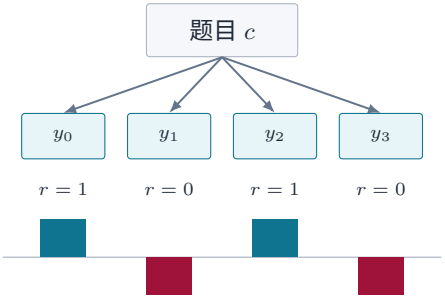
奖励怎样传回每个 token

回答 y 是逐 token 抽出的离散序列，梯度不能穿过抽样，用 score-function 估计：

$$\nabla_\theta \mathbb{E}_{y\sim\pi_\theta}[r] = \mathbb{E}\Big[(r-b)\sum_t \nabla_\theta \log \pi_\theta(y_t \mid y_{<t}, c)\Big]$$

PPO	近端策略优化：另训一个价值网络作基线 b，每个位置的优势不同
GRPO	组相对策略优化：同一题采样一组回答，以组内平均分作基线
DPO	直接偏好优化：不在线采样，直接用“好 / 差”回答对拟合 q^*

score-function 估计　由 $\nabla_\theta \pi_\theta = \pi_\theta \nabla_\theta \log \pi_\theta$ 得到，只要 $\log \pi_\theta$ 可微，离散的 y 也适用；基线 b 不改变期望，只降低方差。



GRPO：答对的整条轨迹各 token 概率被抬高，答错的被压低。

想一想：没人教它验算

RLVR：不示范解题步骤，只让程序判最后答案的对错，对给 1 分，错给 0 分。训练一段时间后，回答越来越长，模型开始自己写出这样的话：

……所以 $x = 12$。
等等，我再验算一下：代回原式，左边是 30，右边是 32，不对。
回到第二步重新算……

示意

问题

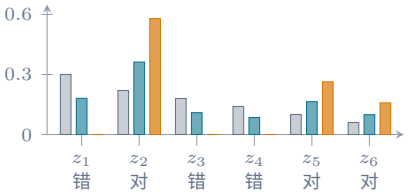
1 没人教过它验算。这种行为是从哪里来的？
2 RL 是教会了模型新的解题方法，还是做了别的事？

推理模型：RLVR 与思维链

回答 $y=(z,a)$：先写推理链 z，再给答案 a。数学、代码可由程序判对错：$r(y)=\mathbf{1}[a \text{ 正确}]$。

- 目标 $\mathbb{E}_{\pi_\theta}[r]=\sum_z \pi_\theta(z\mid c)\,P(a \text{ 对}\mid z,c)$：推理链像隐变量一样被求和；奖励只看答案，不需要标注解题步骤
- 用上一页的 GRPO 训练：答对的整条推理链被抬高
- 训练中回答变长，自发出现验算、回退（OpenAI o1，2024；DeepSeek-R1，2025）。多写的 token 就是额外的串行计算

局限　判题器要可靠；只看答案时，过程可能是碰对的。p_0 几乎不写的推理方式很难被采到，也就很难得到奖励。



六条推理链：灰为 p_0，青为 $\tau=1$，橙为 $\tau\to 0$。p_0 下答对的概率是 0.38

条件概率　代入 RLHF 页的 $q^*=p_0 e^{r/\tau}/Z$，r 只取 0 或 1：

$$q^*(y)\propto p_0(y)\,e^{r(y)/\tau}\xrightarrow{\tau\to 0} p_0(y\mid \text{答对})$$

答对的链按原有相对概率放大 $1/0.38$ 倍，答错的被剔除。

RL 教会了模型什么

回到问题 1　验算、回退在 p_0 的输出里本来就偶尔出现。带验算的链更常答对，奖励把它们一轮轮放大，回答也随之变长。DeepSeek-R1-Zero 的论文把训练中途出现的这种“等等，再想想”称作 aha moment。

回到问题 2　主要是重新分配概率。上一页 $\tau \to 0$ 时 $q^* = p_0(y \mid \text{答对})$：答对的链按原有比例放大，$p_0$ 几乎不写的推理方式很难被采到。实验上也有迹象：允许采样很多次时（pass@k，即 k 次采样中至少一次做对的比例，k 很大），RL 之后的模型能做对的题并不比基座模型多（Yue 等，2025）。

要点

RL 更像是把“偶尔做对”变成“稳定做对”。目标仍是上周的变分自由能：在先验 p_0 附近，按奖励重新加权。

生成模型放回科学计算

同一个模型，两个方向

统计物理里最熟悉的生成模型，就是 Ising（Potts）模型：

$$p(s) = \frac{1}{Z} \exp\Big(\sum_i h_i s_i + \sum_{i<j} J_{ij} s_i s_j \Big)$$

	正问题：有能量，求样本	反问题：有样本，求能量
已知	外场 h 与耦合 J	大量构型 $s^{(1)}, \ldots, s^{(N)}$（模拟、实验、序列）
要求	样本、自由能、关联函数	外场 h 与耦合 J
目标	反向 KL：变分自由能	前向 KL：极大似然
难点	采样：模式多，链混合慢	每步梯度都要从模型采样（负相位）
本节例子	变分自回归网络（VAN）	逆 Ising；蛋白质的直接耦合分析

> **要点**
> 这正是上周“两种学习情形”的两头：手里有的是能量，还是数据。

正问题：变分自回归网络（VAN）

$$q_\theta(s) = \prod_{i=1}^{N} q_\theta(s_i \mid s_{<i}), \qquad s_i = \pm 1$$

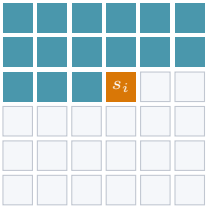
- 每个因子是一个伯努利分布，由带 mask 的网络给出；乘积自动归一化
- 能精确算出 $q_\theta(s)$，又能逐个自旋直接抽样，不需要马尔可夫链

★ 训练：最小化变分自由能

$$F_q = \mathbb{E}_{s\sim q_\theta}\big[E(s) + T\log q_\theta(s)\big] \;\geq\; F$$

$$\nabla_\theta F_q = \mathbb{E}_{s\sim q_\theta}\Big[\big(E + T\log q_\theta - b\big)\nabla_\theta \log q_\theta\Big]$$

没有数据：从 q_θ 抽一批构型，自己算能量，自己更新。



按行编号：s_i（橙）只读前面的自旋（青）

与大模型后训练　就是 RLHF 换一个奖励：$r = -E$，温度 $\tau = T$，参考分布取均匀分布。梯度估计与 GRPO 相同。

原文：16×16 二维铁磁 Ising，自由能明显优于平均场和 Bethe 近似（Wu, Wang & Zhang, PRL 2019）。

想一想：低温下全是向上的自旋

在 $T < T_c$ 训练 VAN（二维铁磁 Ising，无外场）。训练收敛了，变分自由能也很低。可抽出来的构型几乎都是磁化向上，从来没有向下的。



从 q_θ 抽出的构型（青：$+1$，橙：-1）

问题

问题出在哪？

A 网络不够大
B 训练还没收敛
C 目标函数本身就会这样

反向 KL 只抓一个模式

回到问题 C。目标是反向 KL：回到上周“一个高斯拟合两个峰”，反向 KL 落在一个峰上。代价有多大？若 q 恰好是 π 限制在 $m > 0$ 那一半上，$\mathrm{KL}(q\|\pi) = \log 2$，自由能只高 $T \log 2$，与自旋数 N 无关；摊到每个自旋几乎为零，优化器察觉不到。

后果 能量、$|m|$、关联函数看起来都对；$\langle m \rangle$ 却不为零，磁化率等涨落量出错。这个对称破缺来自训练，不是物理。

修补办法

- 强制对称：$q_{\mathrm{sym}}(s) = \frac{1}{2}\left[q_\theta(s) + q_\theta(-s)\right]$，仍可精确计算、直接抽样
- 退火：从高温开始训练，逐步降温。VAN 原文在 Hopfield 模型上这样训练，四个模式都保住了
- 重加权：权重 $w = e^{-E/T}/q_\theta$ 修正偏差；但 q_θ 完全没覆盖的模式，权重也救不回来

反问题：从关联到耦合

手里只有样本。能可靠估计的，是平均值 $\langle s_i \rangle$ 和两两关联 $\langle s_i s_j \rangle$。

★ 最大熵

在约束 $\langle s_i \rangle_p = \langle s_i \rangle_{\text{data}}$、$\langle s_i s_j \rangle_p = \langle s_i s_j \rangle_{\text{data}}$ 下，求熵 $S[p]$ 最大的分布：

$$p(s) = \frac{1}{Z} \exp\Big(\sum_i h_i s_i + \sum_{i<j} J_{ij} s_i s_j \Big)$$

h、J 就是约束的拉格朗日乘子。只约束平均能量时，同样的推导给出玻尔兹曼分布。

★ 极大似然

$$\frac{\partial \overline{\log p}}{\partial J_{ij}} = \underbrace{\langle s_i s_j \rangle_{\text{data}}}_{\text{正相位}} - \underbrace{\langle s_i s_j \rangle_{\text{model}}}_{\text{负相位}}$$

就是上周能量模型的正负相位。收敛时，模型的关联与数据一致。

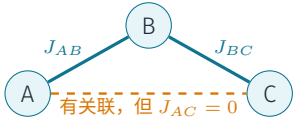
代价　负相位要用 MCMC 从当前模型采样，每更新一步就要采一次。常用近似绕开：平均场给出 $J_{ij} \approx -(C^{-1})_{ij}$（$C$ 为连通关联矩阵，$i \neq j$），或改用伪似然。

蛋白质：从序列推结构，再生成新序列

同一个蛋白家族的上万条同源序列对齐后，每个位置有 21 种状态（20 种氨基酸加空位）。拟合一个 Potts 模型 $J_{ij}(a,b)$。

		i					j		
M	P	K	L	A	V	G	E	R	L
M	A	K	L	S	V	G	E	K	L
M	P	E	I	A	V	N	K	R	L
L	P	E	L	A	I	G	K	R	F
M	S	K	V	A	V	G	E	R	L
M	P	E	L	T	V	D	K	K	L

i 是 K 时 j 多半是 E，反之亦然：两处一起变（共进化）



关联不等于耦合　A 与 B、B 与 C 有直接作用，A 与 C 也会关联。直接看关联矩阵，会把间接关联当成接触；拟合出 J，才能把直接作用分出来。

- **接触预测**：$|J_{ij}|$ 大的位置对，在三维结构里往往挨着。这就是直接耦合分析（DCA；Weigt 等，2009；Morcos 等，2011）
- **生成新酶**：从学到的 Potts 模型抽出新序列去合成，得到有天然酶般催化活性、序列却差别很大的蛋白（Russ 等，Science 2020）

多序列比对里的共进化信号，后来也是 AlphaFold 的重要输入。

数据与评价

数据分布比数据总量更重要

训练目标只看得见经验分布 $\hat{\pi}$。数据由多个来源混合时 $\hat{\pi} = \sum_k \alpha_k \hat{\pi}_k$：改配比就是改目标，总量不变也一样。

数据模式	可能后果	检查
长尾	占 2% 的稀有模式，在平均 loss 里也只占 2% 的权重	按领域、相区、长度分层报告
重复与污染	测试题泄漏进训练集，记忆被当成能力	去重；检查训练与测试的相似度
轨迹相关	同一条 MD 或 MC 轨迹的相邻帧几乎相同；逐帧随机切分，测试成绩虚高	整条轨迹只进一个集合；预测未来时按时间切分
非平稳	换了探测器、调了温度，数据分布跟着变	留出较新的数据单独测试
合成递归	用模型生成的数据训练下一代，稀有模式逐轮丢失：每轮丢 10%，十轮后剩 $0.9^{10} \approx 0.35$	保留真实数据；记录生成器与合成比例

生成模型怎样评价：三个问题分开问

1. 模型给真实测试样本多大概率？
   测试集 NLL；语言模型报告困惑度
   $\mathrm{PPL} = \exp(\text{平均 token NLL})$
2. 单个生成样本是否可信？
   保真度（precision）
3. 大量样本能否覆盖全部模式和尾部？
   覆盖度（recall）

三者互不替代　似然高不等于样本好；样本个个逼真，也可能只来自一个模式。只保留高分样本，保真度上升，覆盖度下降。

指标	比较什么；局限
NLL / PPL	给真实数据的概率；不直接说明抽出的样本有没有用
FID	真实图像与生成图像在一个分类网络特征里的均值和协方差之差；把保真度和覆盖度混成一个数
Precision / Recall	在特征空间里分别估计保真度和覆盖度；依赖特征和邻域的选择

后两项依赖图像分类网络的特征；分子、自旋构型要换成与对象相关的物理量。

物理数据：“看起来像”只是最低层的证据

1. **可观测量的分布**：能量、磁化、序参量、密度
2. **关联函数**：两点关联、结构因子、径向分布函数
3. **约束**：守恒律、归一化、对称性、边界条件（前面 VAN 的 $\mathbb{Z}_2$ 对称就是一例）
4. **独立基准**：精确解、MC/MD 模拟、实验；有效样本数、模式覆盖、稀有事件

重要性权重　能精确算 q_θ 的模型（自回归、Flow）可以重加权：$w = \tilde{\pi}/q_\theta$。有效样本数 $\mathrm{ESS} = (\sum w)^2 / \sum w^2$ 直接说明 q_θ 离目标有多远；ESS 远小于样本数时，加权平均也不可信。

要点

生成模型给出的是候选。样本来自哪里、怎样切分、在哪个分布上评价，都要记录下来；只有通过独立评估和物理检验的输出，才能写进结论，或回流成下一版的训练数据。

全课回顾：六条路线

	分解　自回归	变换　Flow	隐变量　VAE
算出 $p(x)$	$p(x) = \prod_t p(x_t \mid x_{<t})$ 精确似然；逐个采样，串行 链式法则	$x = f_\theta^{-1}(z),\ z \sim \mathcal{N}(0, I)$ 精确似然；可逆结构受限 随机变量的变量替换	$p(x) = \int p(z)\, p_\theta(x \mid z)\, dz$ 只能优化下界 边缘化、Jensen 不等式
绕开归一化	**对抗　GAN** $x = G_\theta(z)$ 不给密度，只比较样本 两样本检验	**能量　EBM** $p(x) \propto e^{-E_\theta(x)}$ 未归一化；靠马尔可夫链采样 玻尔兹曼分布、MCMC	**去噪　扩散** $s_\theta(x, t) \approx \nabla_x \log p_t(x)$ 学习各噪声水平的 score 高斯卷积、条件期望、SDE

Transformer 是第一张卡片里的条件分布网络；扩散模型的去噪网络也常用它。大模型的后训练和 VAN 都回到上周的自由能。

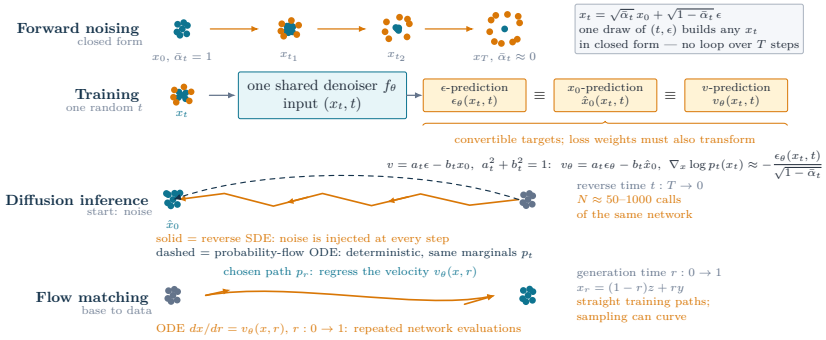
全课小结

带走的几件事

1. 生成模型用样本（或能量）学一个高维分布，并从中抽样；难点集中在归一化常数 Z
2. 训练目标都是分布之间的差距：极大似然（包括逆 Ising）是前向 KL；变分自由能、RLHF 和 VAN 是反向 KL；GAN 是 JSD；score matching 是 Fisher 散度
3. 扩散把一次困难的采样拆成许多次去噪；噪声回归学到的后验平均，恰好就是 score
4. 大语言模型 = 链式法则 + Transformer + 极大似然，再用带 KL 约束的奖励优化做对齐；生成时的瓶颈是每步读一遍 KV cache
5. 生成模型给出的是候选：先看清数据分布和评价方式，再用物理量、约束和独立基准检验

附录

附录：一个统一的图景



要点

DDPM 学噪声（等价于 score），用随机的反向转移生成；同一个网络也可以沿概率流 ODE 确定性地大步采样（见后面的 DDIM 页）；Flow Matching 直接学选定路径上的速度。在高斯路径上，噪声、干净样本、score 与速度可以互相换算，差别在路径、损失权重和求解器。

附录：为什么回归条件速度就够了

★ 三步

条件路径：固定数据点 y，路径 $x_r = (1-r)z + ry$ 的速度 $u_r(x \mid y) = \frac{y-x}{1-r}$ 已知，条件密度 $p_r(x \mid y)$ 满足连续性方程。

叠加：令 $v_r(x) = \mathbb{E}[y - z \mid x_r = x]$，则

$$p_r(x)\, v_r(x) = \int p_{\text{data}}(y)\, p_r(x \mid y)\, u_r(x \mid y)\, dy$$

对 y 积分条件连续性方程，得 $\partial_r p_r + \nabla \cdot (p_r v_r) = 0$：$v_r$ 把 $\mathcal{N}(0, I)$ 恰好搬运成 p_{data}。

损失分解：

$$\mathbb{E}\big\|v_\theta - (y - z)\big\|^2 = \mathbb{E}\big\|v_\theta - v_r(x_r)\big\|^2 + C$$

同一个技巧在正文出现了两次：把不可算的边缘目标换成可算的条件目标。

- **扩散**：边缘 score $\nabla \log p_t(x)$ $\rightarrow$ 条件 score $-\epsilon / b_t$
- **流**：边缘速度 $v_r(x)$ $\rightarrow$ 条件速度 $y - z$

条件期望　均方误差下，对条件目标回归与对边缘目标回归只差一个与参数无关的常数，最优解相同。

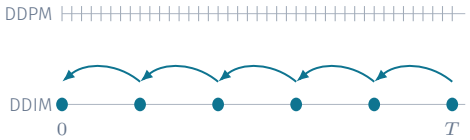
附录：DDIM（去噪扩散隐式模型），同一个网络，更少的步数

在当前 x_t 调用一次网络，先估计干净样本，再按更早时刻 $s < t$ 的系数重新组合：

$$\hat{x}_0 = \frac{x_t - b_t\,\epsilon_\theta(x_t, t)}{a_t},$$

$$x_s = a_s \hat{x}_0 + b_s\,\epsilon_\theta(x_t, t)$$

- 训练不变，只改采样器；不再注入新噪声，相当于沿概率流 ODE 走大步
- 1000 步训练调度，生成时只取 50 个时刻
- 更好的 ODE 求解器、蒸馏等方法还能进一步压到几步



要点

扩散的计算成本主要来自步数；训练出的 score 网络可以配不同的采样器。

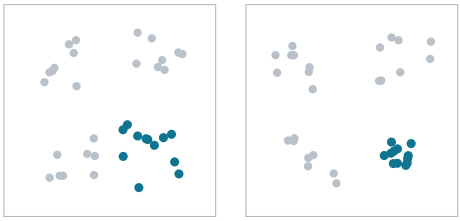
附录：条件生成，classifier-free guidance

训练时随机丢掉一部分样本的条件 c（如文本描述），同一个网络既能给出 $\epsilon_\theta(x_t, t, c)$，也能给出 $\epsilon_\theta(x_t, t, \varnothing)$。采样时

$$\epsilon_{\text{guide}} = \epsilon_{\text{uncond}} + w\,(\epsilon_{\text{cond}} - \epsilon_{\text{uncond}})$$

贝叶斯公式

$\nabla \log p(x \mid c) = \nabla \log p(x) + \nabla \log p(c \mid x)$。引导相当于从 $p(x)\,p(c \mid x)^w$ 采样：$w > 1$ 把似然放大，样本更贴合条件，多样性下降。



青色：条件 c 下的样本；灰色：其他类别

文生图模型中的“提示词相关度”参数就是 w；每步需要两次网络调用。

附录：怎样对“抽样”求梯度

ELBO 里有 $\mathbb{E}_{z\sim q_\phi}[\cdot]$，而抽样分布本身含有待训练的 ϕ。两条路径：

	重参数化	score-function
写法	$z=\mu_\phi+\sigma_\phi\odot\epsilon$，$\epsilon\sim\mathcal{N}(0,I)$	$\nabla_\phi\mathbb{E}_{q_\phi}[h]=\mathbb{E}_{q_\phi}\left[h\,\nabla_\phi\log q_\phi(z)\right]$
梯度从哪里来	沿 $\epsilon\to z\to h$ 的计算图回传	用得分 h 加权对数概率的梯度
要求	抽样可写成可微变换	只需 $\log q_\phi$ 可微；z 可以离散
方差	通常较低	通常较高，常减去一个基线
用在哪里	VAE、GAN	强化学习、大模型 RLHF、VAN

正态随机变量的标准化 $z\sim\mathcal{N}(\mu,\sigma^2)$ 等价于 $z=\mu+\sigma\epsilon$。把随机性挪到与参数无关的 ϵ 上，z 就对 μ,σ 可微。